PU1. Multiprocess Programming

Advanced Vocational Training Cycle

Cross-Platform Application Development

Processes and Services Programming

2026-2027 academic year

Contents

1. Programs, processes and services
2. Multiprogramming
3. Concurrent, parallel and distributed programming
4. Process components
5. Process Control Block (PCB)
6. Process states
7. Process context switching
8. Process scheduling
9. Threads
10. Process management
11. Inter Process Communication (IPC)
12. Synchronization in IPC

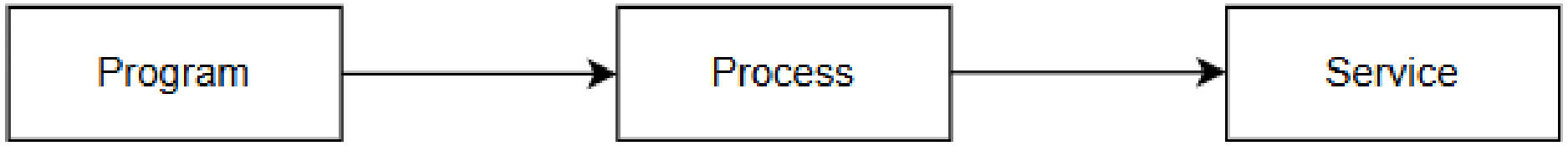
1. Programs, processes and services

A **program** is a set of algorithms (ordered instructions) that can be executed by the Operating System.

When a program is started, the OS loads it into memory and it becomes a process. A **process** is the execution of a program performing all the tasks mentioned in it.

A **service** is a process running in background that provides functionality to the system or other programs (for example, web server, database, antivirus).

Program → code stored on disk
Process → executing program in memory
Service → background process providing functionality



2. Multiprogramming

A **monoprocessor** system has only one processor to perform the tasks.
A **multiprocessor** system consists of more than one processor or a single processor with more than one core (multicore).

A multiprocessor system can execute several processes at the same time.
The fact that an Operating System manages the execution of concurrent processes on a single-processor system is the basis of **multiprogramming**.

The simultaneous execution of processes is also called **concurrency**.

3. Concurrent, parallel and distributed programming

Concurrent programming is when different tasks (processes) are executed simultaneously on a computer device.

When concurrent programming is performed on a multiprocessor system, we refer to it as **parallel programming**.

Distributed programming is when the processes of concurrent programming are executed on a network of processors or on a distributed system, which is a system of independent computers seen as a single computer (clustering).

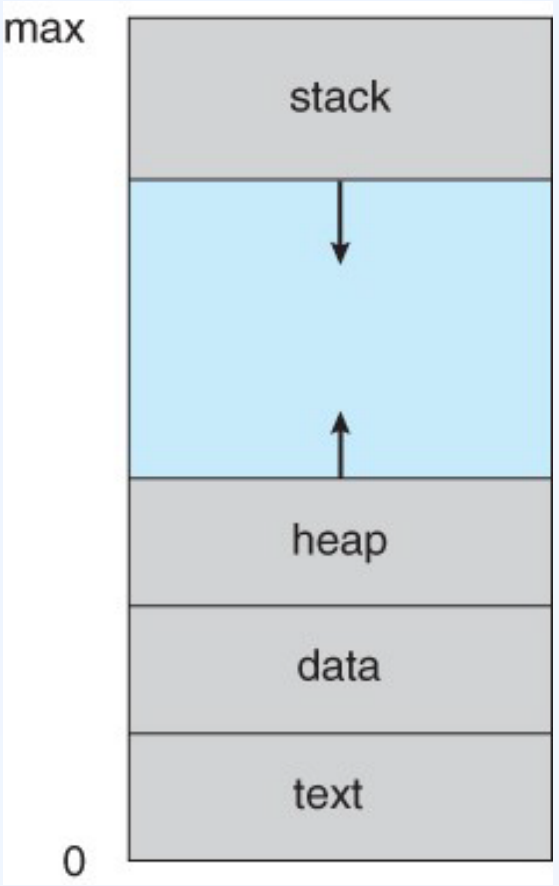
4. Process components

A process in memory has the following sections:

- **Text**: keeps the compiled program code and executable instructions. It is loaded from non-volatile storage when the process is launched.
- **Data**: stores global and static variables, allocated and initialized before executing main.
- **Heap**: used for dynamic memory allocation via calls to new, delete, malloc, free, etc.
- **Stack**: used for local variables, function parameters and return values. Space is reserved when declared and freed when variables go out of scope.

4. Process components

The stack and heap grow from opposite ends of the process free space. If they meet, a stack overflow or memory-allocation failure may occur. Some languages use garbage collectors to prevent this situation.



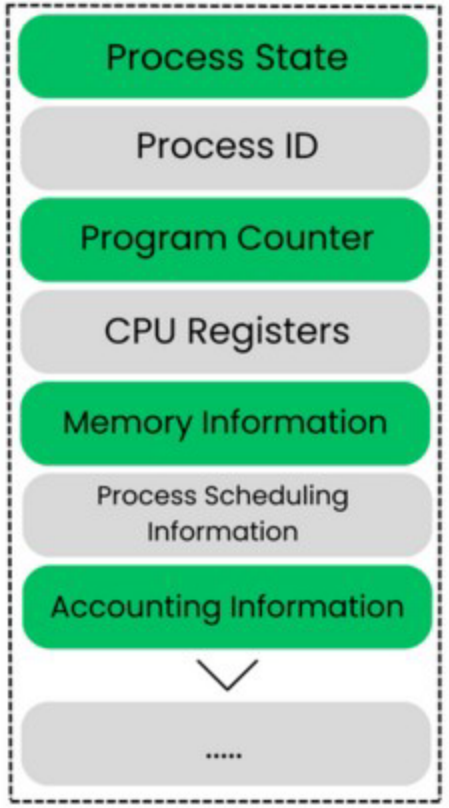
5. Process Control Block (PCB)

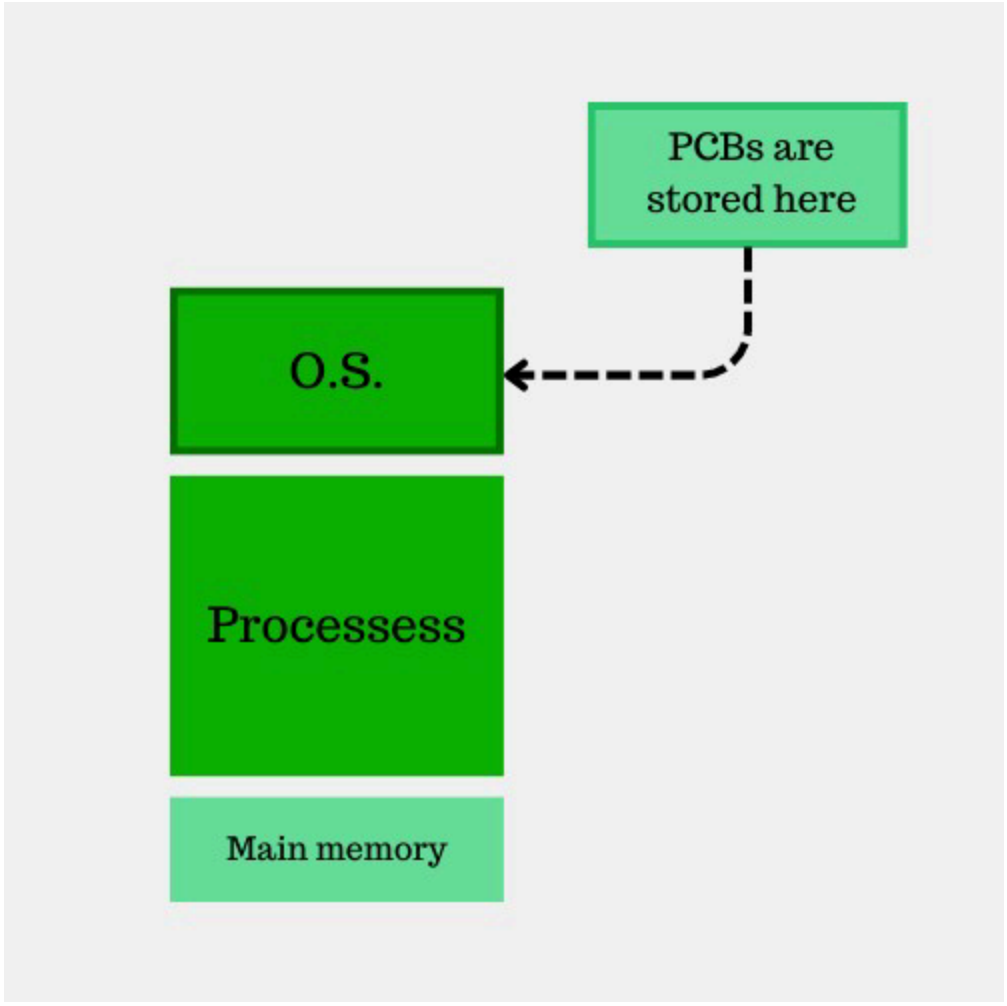
The Process Control Block is a structure for each process with process-specific information.

- **Process state**: current state of the process used to manage and schedule it.
- **Process ID**: unique identifier assigned by the OS when it is created.
- **Program Counter and CPU registers**: point to the next instruction and save/restore CPU context during switching.
- **CPU scheduling information**: priority and pointers to scheduling queues.
- **Memory-management information**: page tables, segment tables, memory limits, etc.
- **Accounting information**: CPU time consumed, limits, etc.
- **I/O status information**: devices allocated, open file tables, etc.

5. Process Control Block (PCB)

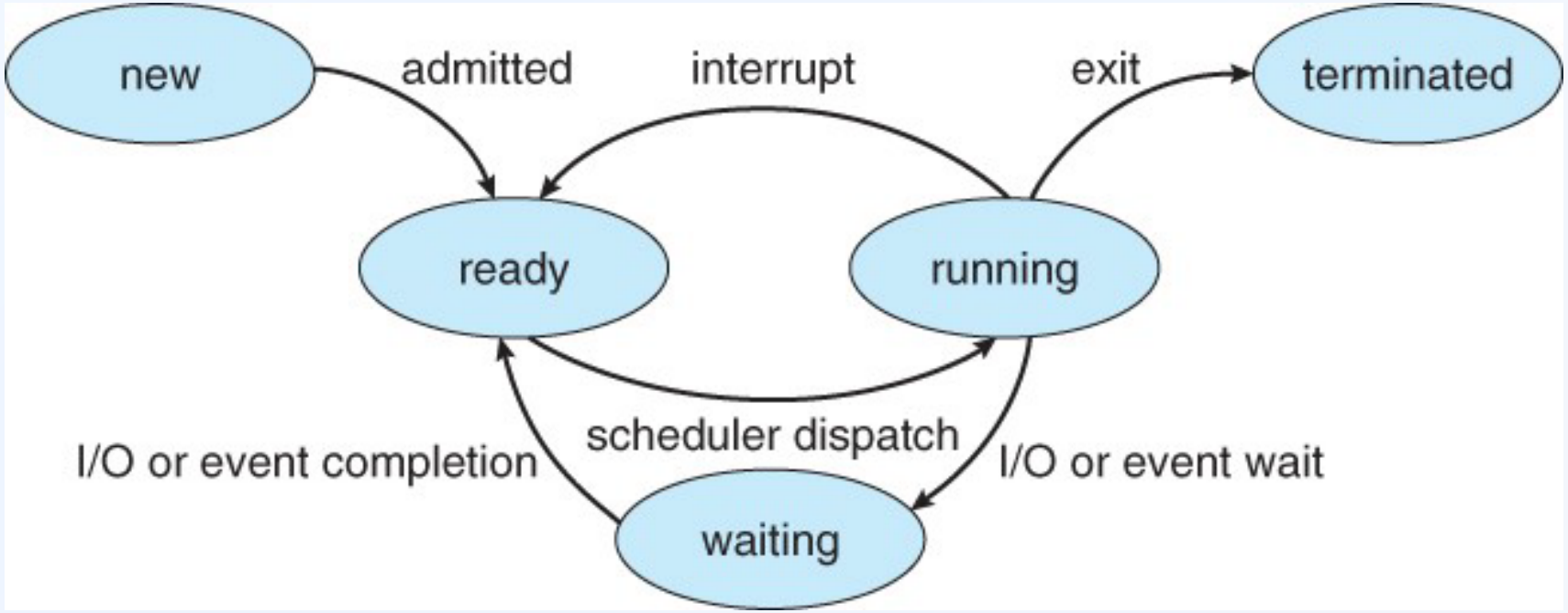
The OS maintains a Process Table with all active Process IDs (PIDs) and their PCB.





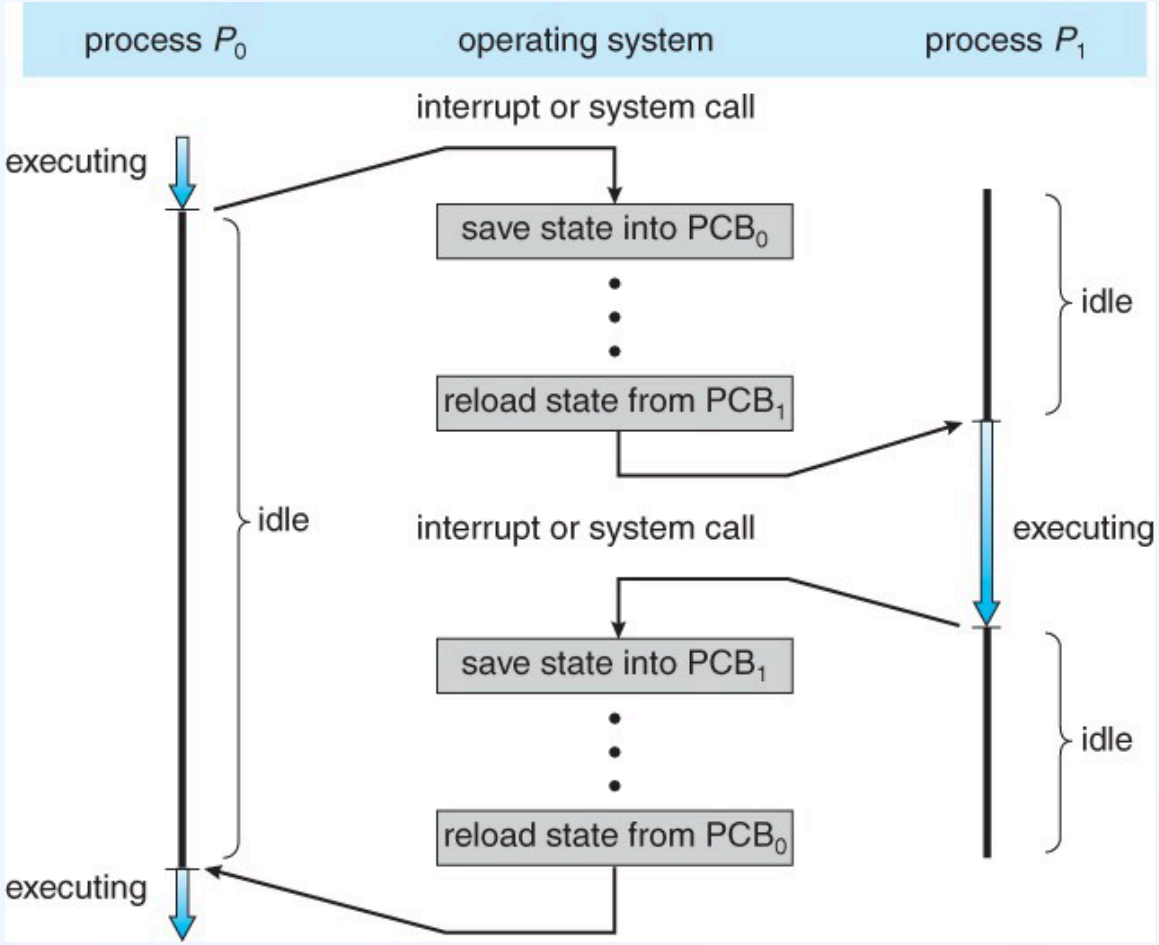
6. Process states

- **New**: the process is being created.
- **Ready**: the process has all required resources, but the CPU is not currently executing it.
- **Running**: the CPU is executing its instructions.
- **Waiting**: the process is blocked waiting for a resource, event, I/O operation or child process.
- **Terminated**: the process has completed.



7. Process context switching

Process switching is the mechanism by which the OS saves the state of the current process in its PCB and restores the state of another process scheduled to run. This allows multiple processes to share the CPU efficiently and gives the impression of simultaneous execution.



8. Process scheduling

The process scheduling system has two main objectives:

- To keep the CPU busy at all times.
- To deliver acceptable response times for all programs.

There are two main schedulers:

- **Short-term scheduler** or **CPU scheduler**: selects ready processes to execute.
- **Medium-term scheduler**: manages swapping.

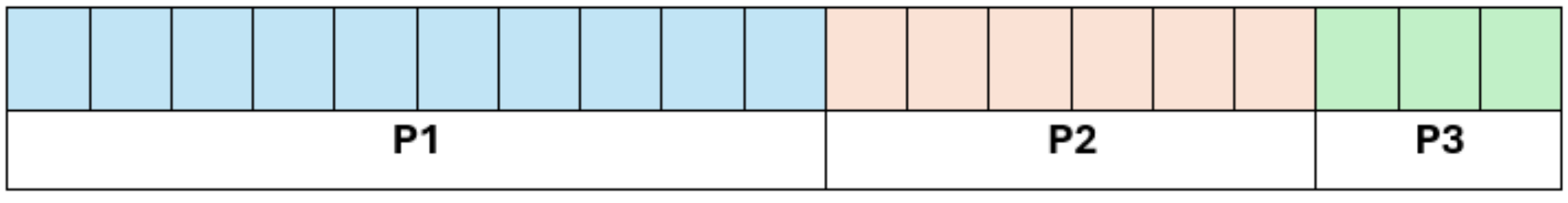
There are some queues for processes: ready queue, I/O queue, blocked queue, etc.

8.1. Process scheduling algorithms. FCFS

FCFS (First Come, First Served)

Executes the first process, then the second, and so on (commonly called FIFO: First In, First Out).

Process	Arrival time	CPU burst time
P1	0	10
P2	2	6
P3	4	3



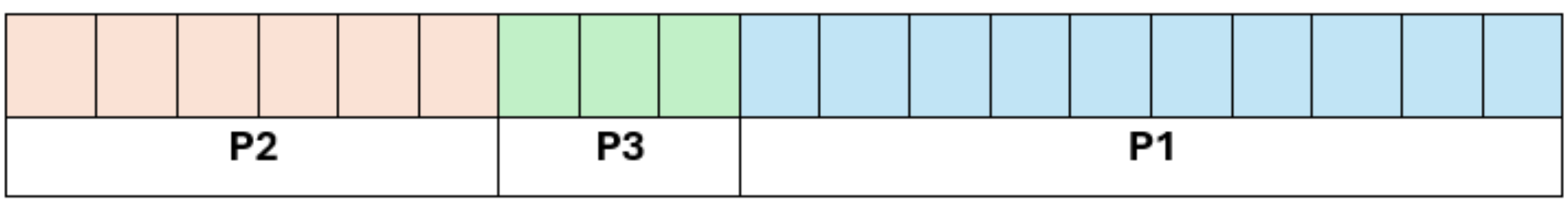
8.2. Process scheduling algorithms. SJF

SJF (Shortest Job First)

Executes the shortest process, or the first arrived when the lengths are equal.

This algorithm may suffer from starvation: a long process may not be executed if shorter ones keep arriving.

Process	Arrival time	CPU burst time
P1	0	10
P2	0	6
P3	2	3

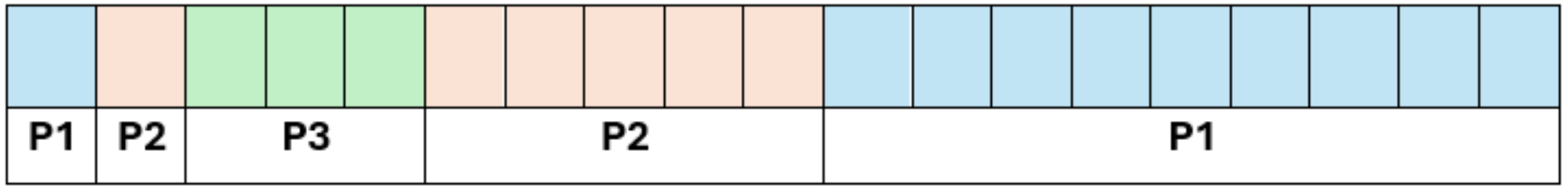


8.3. Process scheduling algorithms. SRTF

SRTF (Shortest Remaining Time First)

It behaves like SJF but is preemptive: a running process can be interrupted if a new shorter job arrives. It gives better responsiveness than SJF in dynamic workloads, but it is more complex to implement.

Process	Arrival time	CPU burst time
P1	0	10
P2	1	6
P3	2	3



8.4. Process scheduling algorithms. Priority algorithm

Each process has a priority value; the higher the priority, the earlier it is executed.

The priority can be assigned:

- Statically
- Dynamically

The algorithm can be implemented:

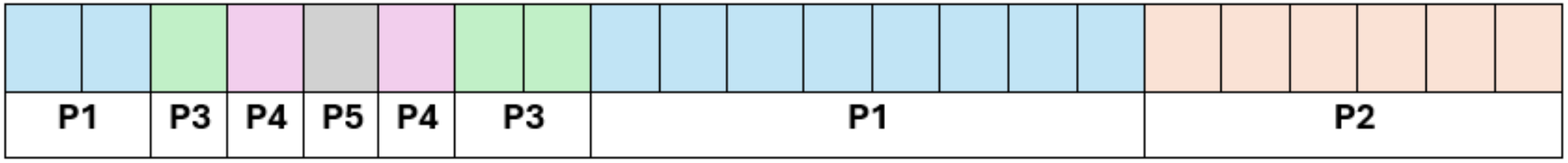
- Preemptive
- Non-preemptive

Priority scheduling is simple and effective, but it can also lead to starvation unless aging is used.

8.4. Process scheduling algorithms. Priority algorithm

Priority algorithm: Static priority and preemptive version example:

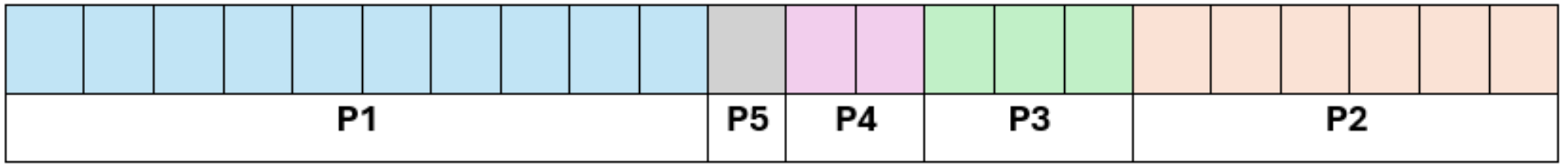
Process	Arrival time	CPU burst time	Priority
P1	0	10	2
P2	1	6	1
P3	2	3	3
P4	3	2	4
P5	4	1	5



8.4. Process scheduling algorithms. Priority algorithm

Priority algorithm: Static priority and non-preemptive version example:

Process	Arrival time	CPU burst time	Priority
P1	0	10	2
P2	1	6	1
P3	2	3	3
P4	3	2	4
P5	4	1	5



8.5. Process scheduling algorithms. RR

RR (Round Robin)

Each process receives a time slice or quantum for execution in a cyclic turn.

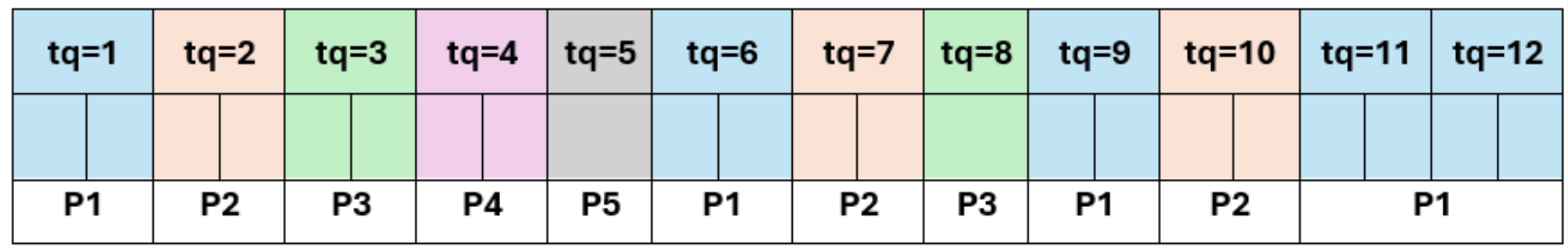
This is a fair policy, especially in interactive systems, but it can increase response time if the quantum is too small.

In the next slide we can see an example of quantum=2.

8.5. Process scheduling algorithms. RR

RR (Round Robin)

Process	Arrival time	CPU burst time
P1	0	10
P2	1	6
P3	2	3
P4	3	2
P5	4	1



8.6. Process scheduling algorithms. Compare

It is possible to combine algorithms. For example, Round Robin can be combined with FCFS.

We can compare algorithms by time waiting to be executed and time since the process arrival and ending:

Process	FCFS	SJF	STRF	RR
Delay time	6,66	3,6	0	2
Return time	13	10,6	11	61

SJF generally has the best average waiting time, while RR usually offers very good response time but a higher return time due to the quantum time.

9. Threads

A **thread** is a single sequence stream within a process. Threads are also called **lightweight processes** because they have some of the properties of processes.

Each thread belongs to exactly one process.

Threads in the same process share memory space, while processes usually run in separate memory spaces. Therefore, threads are not independent from one another as processes are.

Threads share code, data and OS resources such as open files and signals.

We will work with threads in the next unit.

10. Process management

A process can be created by the OS, another process or by the user.

The `fork()` function is used to create a process.

- `fork()` returns the child's PID to the parent process.
- It returns `0` when called by the child process itself.

Processes are independent and do not share memory, so they exchange messages to communicate with each other.

10.1. Process management: Create a process in Python

```
# fork only works in Unix/macOS
# Child process creation
import os

def child():
    print('\n>>>>>>>>>> New child created with pid %d is going to finish <<<<<' % os.getpid())
    os._exit(0)

def parent():
    while True:
        newpid = os.fork()
        if newpid == 0:
            child()
        else:
            pids = (os.getpid(), newpid)
            print("Parent: %d, Child: %d\n" % pids)
            reply = input("Press 'y' to create a new process")
            if reply != 'y':
                break

parent()
```

10.1. Process management: Create a process in Python

```
# fork only works in Windows
from multiprocessing import Process
import os

def child():
    print("Parent: %d, Child: %d\n" % (os.getppid(), os.getpid()))
    os._exit(0)

def parent():
    while True:
        p = Process(target=child)
        p.start()
        print("\nNew child created ", p.pid)
        p.join()
        reply = input("Press 'y' to create a new process\n")
        if reply != 'y':
            break

if __name__ == '__main__':
    parent()
```

10.2. Process management: launch (run)

The easiest way to launch a process is using the static method `run()` of its child process `subprocess`.

`run()` receives:

- the executable file path or executable name (in the case it is included on the system path)
- the executable with a list of parameters

Examples:

```
import subprocess

subprocess.run(['Notepad.exe'])
subprocess.run(["ping", "www.google.com", "-n", "5"])
```

10.2. Process management: launch (run) with examples

For Linux:

```
import subprocess

try:
    subprocess.run(["ping", "www.google.com", "-c", "5"])
except subprocess.CalledProcessError as e:
    print(e.output)
```

For Windows:

```
try:
    subprocess.run(['Notepad.exe'])
    subprocess.run(['c:/windows/notepad.exe'])
    subprocess.run(['Notepad.exe', 'texto.txt'])
    subprocess.run(["ping", "www.google.com", "-n", "5"])
except subprocess.CalledProcessError as e:
    print(e.output)
```

10.3. Process management: access process information

If we keep a reference to the process returned by `run()`, we can access its properties and state.

To access system processes, we can install the `psutil` package:

Windows:

```
pip install psutil
```

```
import psutil

for proc in psutil.process_iter():
    process_name = proc.name()
    process_id = proc.pid
    print(process_name, ' :: ', process_id)
```

10.3. Process management: access process information

```
import subprocess
import time

SW_SHOWMAXIMIZED = 3

def create_process():
    try:
        info = subprocess.STARTUPINFO()
        info.dwFlags |= subprocess.STARTF_USESHOWWINDOW
        info.wShowWindow = SW_SHOWMAXIMIZED
        proc = subprocess.Popen('notepad.exe', startupinfo=info)
        return proc
    except subprocess.CalledProcessError as e:
        print(e.output)

p = create_process()
print("This process PID is: " + str(p.pid))
time.sleep(5)
```

10.4. Process management: access current process and priority

```
import os
import sys
import subprocess
import psutil

def current_process():
    return psutil.Process(os.getpid())

def is_windows():
    try:
        sys.getwindowsversion()
    except AttributeError:
        return False
    else:
        return True

print(current_process().name())
print(current_process().cwd())
print(current_process().nice())

if is_windows():
    subprocess.check_output("wmic process where processid=\"" + str(os.getpid()) + "\" CALL setpriority \"below normal\"")
else:
    os.nice(1)

print(current_process().nice())
a = input()
```

10.5. Process management: removal

A process can finish by itself after completing its tasks, or it can be terminated by the OS or an authorized user.

The method used to terminate a process is `kill()`. In Unix systems, it sends the `SIGKILL` signal; in Windows, it invokes the `terminate()` function.

The deletion of a process is asynchronous. It may still exist when the next instruction is executed. To know if it has terminated, we can use `poll()`, which returns `None` when the process has been completely removed.

10.5. Process management: removal

This code deletes all notepad.exe processes:

```
import psutil

for proc in psutil.process_iter():
    try:
        process_name = proc.name()
        if process_name == "notepad.exe": # or firefox if we are using Linux
            pid = proc.pid
            print("Killing process: ", process_name, ' ::: ', pid)
            proc.kill()
    except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
        print("error")
```

10.6. Process management: asynchrony

Programs usually run **synchronously**, and some methods like `run()` block the program until they complete.

Asynchronous programming executes instructions independently of the blocking operation and does not wait for it to finish. Some mechanisms are:

- `async def` : defines a function with asynchronous operations.
- `await()` : indicates that the completion of the function is not awaited immediately.
- `asyncio.run()` : starts the execution of an asynchronous function.

10.6. Process management: asynchronous example

```
#For Windows
import asyncio

async def greet(name):
    await asyncio.sleep(2)
    print(f"Hello, {name}!")

async def main():
    await asyncio.gather(
        greet("Silvia"),
        greet("Paco"),
        greet("Isabel")
    )

asyncio.run(main())
```

Although each greeting waits 2 seconds, they all run concurrently, so the program takes just over 2 seconds in total.

11. Inter Process Communication (IPC)

To exchange objects between processes we can use **queues** and **pipes**.

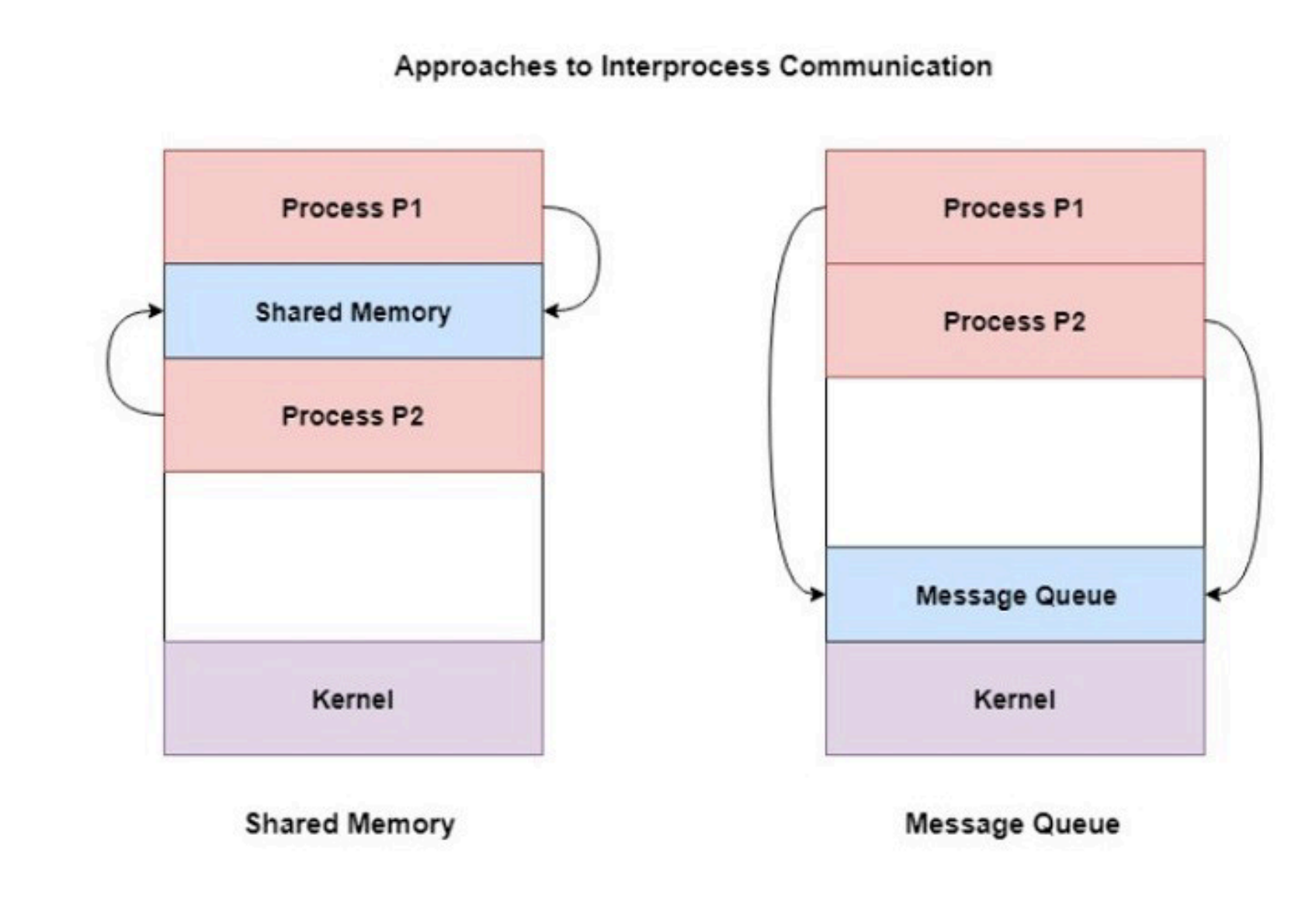
If we need to share state between processes, we can use **shared memory** and a server process controlled by a manager.

If the processes are distributed, we can use **sockets** to communicate them.

There is another way to use the process input and output through a console:

- `stdout` : standard output
- `stdin` : standard input
- `stderr` : standard error

11. Inter Process Communication (IPC)



12. Synchronization in IPC

Synchronization is either provided by the inter-process control mechanism or handled by the communicating processes.

Its goals are:

- **Preventing race conditions**: do not access shared data at the same time.
- **Mutual exclusion**: only one process in the critical section at a time.
- **Process coordination**: wait for specific conditions (for example, producer-consumer).
- **Deadlock prevention**: avoid indefinite blocking.
- **Safe communication**: data/messages are sent, received and processed in order.
- **Fairness**: prevent starvation by fair access to resources.

12.1. Synchronization problems in IPC

Producer-Consumer

A producer introduces data in a buffer and a consumer takes it. It is important to ensure the buffer does not overflow (too much data in the buffer) or underflow (buffer empty with nothing to consume).

Readers-Writers

There are multiple readers that can access a resource concurrently, but only one writer can access it exclusively. Synchronization is needed to avoid data corruption. The problem is caused by the concurrent access between writer and readers.

12.2. Synchronization primitives in Python

The `multiprocessing` module contains equivalents of all synchronization primitives from `threading`, which will be studied in PU2.

Some of them are:

- **Lock**: Only one process can modify or access a specific part of a system, data or program at a time.
- **Semaphore**: Is a variable that controls the access to a common resource by multiple processes. It can be binary semaphore and counting semaphore
- **Event**: Is an object that processes can wait for or signal, coordinating actions by establishing a condition for proceeding.
- **Barrier**: It does not allow individual processes to proceed until all the processes reach it.

12.2.1. Lock example

```
import multiprocessing

def worker_function(lock, shared_value):
    with lock: # Process informs it is inside the critical section
        print(f"Process {multiprocessing.current_process().name} is inside the critical section.")
        shared_value.value += 1 # Process increments the shared value and tells it
        print(f"Process {multiprocessing.current_process().name} has incremented the value to {shared_value.value}")

if __name__ == '__main__': # Create a shared value inicialized to 0 and a lock
    shared_value = multiprocessing.Value('i', 0)
    lock = multiprocessing.Lock()
    processes = [] #array to keep the processes

    for _ in range(3): # Create 3 processes in the array
        process = multiprocessing.Process(target=worker_function, args=(lock, shared_value))
        processes.append(process)
        process.start()

    for process in processes:
        process.join() # Wait for all processes to finish

    print("Final shared value:", shared_value.value) # Print the final value of the shared variable
```

Only one process can execute the critical section at a time.

12.2.2. Semaphore example

```
import multiprocessing
import time

def worker_function(semaphore, resource_id):
    with semaphore:
        print(f"Process {multiprocessing.current_process().name} is using resource {resource_id}")
        time.sleep(5)
        print(f"Process {multiprocessing.current_process().name} has finished using resource {resource_id}")

if __name__ == '__main__':
    semaphore = multiprocessing.Semaphore(2) # Allow 2 concurrent processes
    processes = [] #Array to keep the processes

    for i in range(5): # Create 5 processes in the array
        process = multiprocessing.Process(target=worker_function, args=(semaphore, i))
        processes.append(process)
        process.start()

    for process in processes:
        process.join() # Wait for all processes to finish

    print("All processes have finished.")
```

Semaphore(1) behaves like Lock() because it allows only one process in the critical section.

12.2.3 Event example

```
import multiprocessing
import time

def worker_function(event):
    print(f"Process {multiprocessing.current_process().name} is awaiting the event.")
    event.wait() # Process waits until the event is set
    print(f"Process {multiprocessing.current_process().name} has received notification and is proceeding.\n")

if __name__ == '__main__':
    event = multiprocessing.Event()
    processes = [] # Array to keep the processes

    for _ in range(3): # Create 3 processes in the array
        process = multiprocessing.Process(target=worker_function, args=(event,))
        processes.append(process)
        process.start()

    print("Main process is sleeping for 3 seconds.")
    time.sleep(3)
    print("Main process is signaling the event.")
    event.set() # Waiting processes are notified to continue

    for process in processes:
        process.join() # Wait for all processes to finish

    print("All processes have completed.")
```


Processes wait (event.wait()) until main process (parent) executes event.set()

12.2.4. Barrier example

```
import multiprocessing
import time

def worker_function(barrier, number):
    value = number * 2 # Simulate some computation
    print(f"Process {multiprocessing.current_process().name} is in the barrier with value {value}")
    time.sleep(value) # Simulate variable processing time
    barrier.wait() # Process waiting until the barrier is lifted (all processes reach this point)
    print(f"Barrier lifted, Process {multiprocessing.current_process().name} finished and leaves")

if __name__ == '__main__':
    barrier = multiprocessing.Barrier(5 + 1) # Number of processes waiting the barrier lifts
    processes = [] #Array to keep the processes

    for i in range(5): # Create 5 processes in the array
        process = multiprocessing.Process(target=worker_function, args=(barrier, i))
        processes.append(process)
        process.start()

    print("Main process waiting for all results...")
    # Next three lines include the main process in the barrier (if not needed, set Barrier(5) above)
    barrier.wait() # Main process also waits at the barrier
    print("Barrier lifted, Main process continues execution.")

    for process in processes:
        process.join() # Parent waits for all processes to finish

    print("All processes have finished.")
```


It is possible to have a barrier(3) and 5 processes, two of them will not do barrier.wait()

12.3. Synchronization in IPC

Multiprocessing class in Python contains equivalents of all the synchronization primitives from Threading, which will be studied in PU2.
Some of them are:

- **Queue**: it is a data structure with automatic synchronization due to the use of internal locks. It can be used for multiple processes
- **Pipe**: a pipe only connects two processes, each at one of its ends. Communication can be in one direction (simplex) or in both directions (duplex)

Pipe returns a pair of connections (conn1, conn2) to use the pipe. If it is simplex, conn1 is for receiving and conn2 is used to send data.

12.3.1. IPC communication mechanisms. Queue

```
from multiprocessing import Process, Queue

def sender(q):
    # The child process put data in the queue
    msg_sent = "Hello, this is the message sent"
    print(f"Sender process (child) sent: {msg_sent}")
    q.put(msg_sent)

if __name__ == "__main__":
    q = Queue() # Creates the queue
    p = Process(target=sender, args=(q,)) # Creates child process (sender)
    p.start()
    msg_received = q.get() # Parent process retrieves data from the queue
    print(f"Process (parent) received: {msg_received}")
    p.join() # Waits for the child process to end
```

Processes put and get data to and from the queue (stores data).

12.3.2 IPC communication mechanisms. Pipe

```
from multiprocessing import Process, Pipe

def sender(conn):
    # The child process send data via its connection
    data_to_send = "Hello, this is the message sent"
    conn.send(data_to_send)
    print(f"Sender process (child) sent: {data_to_send}")
    conn.close() # We better close the connection when done

if __name__ == "__main__":
    # Creates the pipe. rcv_pipe will be used by parent process to receive data
# snd_pipe will be used by child process to send data
    rcv_pipe, snd_pipe = Pipe()
    # Creates child process with the sender end of the pipe
    p = Process(target=sender, args=(snd_pipe,))
    p.start()
    # Parent process receives data from its end of the pipe
    received_data = rcv_pipe.recv()
    print(f"Receiver process (parent) received: {received_data}")
    p.join()# Waits for the child process to end
    rcv_pipe.close() # Closes parent process connection to pipe
```


Processes use the pipe as a channel, it doesn’t store any data is an end to end communication.